



**WYDZIAŁ
ELEKTROTECHNIKI
I INFORMATYKI**
POLITECHNIKI RZESZOWSKIEJ

Bezpieczeństwo aplikacji Webowych

Aplikacja dla ośrodków szkolenia kierowców

—

<https://github.com/Skalik2/OSK---app>

Autorzy:

Jakub Portas, Szymon Jabłoński, Mateusz Skali
W Gorącej Wodzie Co.

Rzeszów, 2026

Spis treści

1. Wprowadzenie	3
2. Definition and design	3
2.1. Architektura systemu	3
2.2. Kontrola dostępu (RBAC) - Przypadki użycia	4
2.3. Model danych	6
2.4. Potencjalne zagrożenia	7
3. Software supply chain failures	9
3.1. Zestawienie wykorzystanych technologii	9
3.2. Analiza historycznych podatności	9
4. Implementacja aplikacji	11
4.1. Autentykacja, zarządzanie hasłami i RBAC	11
4.2. Autoryzacja i ochrona zasobów	11
4.3. Komunikacja między mikroservisami i weryfikacja tokenów	12
4.4. Integracja z zewnętrznym API pogodowym	13
4.5. Warstwa bazy danych (Security & Safety)	14
4.6. Implementacja testów automatycznych	14
4.6.1. Izolacja środowiska testowego	15
4.6.2. Testowanie mikroservisów autentykacji	15
4.6.3. Testy integracyjne przepływu pracy	15
5. Wdrożenie projektu	17
5.1. Środowisko bazodanowe (PostgreSQL i pgAdmin)	17
5.2. Serwer backend (FastAPI)	17
5.3. Aplikacja Frontend (React)	18
5.4. Testy aplikacji (pytest)	18
6. Podsumowanie i wnioski końcowe	19
Literatura	20

1. Wprowadzenie

Głównym celem niniejszego projektu było zaplanowanie, zaprojektowanie i wstępne wdrożenie aplikacji webowej do zarządzania ośrodkami szkolenia kierowców, opierającej się na architekturze Frontend, Backend na bazie mikroserwisów, bazie danych RDBMS oraz integracji z zewnętrznym API. Ważnym elementem projektu była analiza tworzonej aplikacji pod kątem bezpieczeństwa i niezawodności (security oraz safety). W tym raporcie przedstawiono szczegóły dotyczące architektury i założeń aplikacji, jej implementacji oraz scenariusza wdrożenia.

2. Definition and design

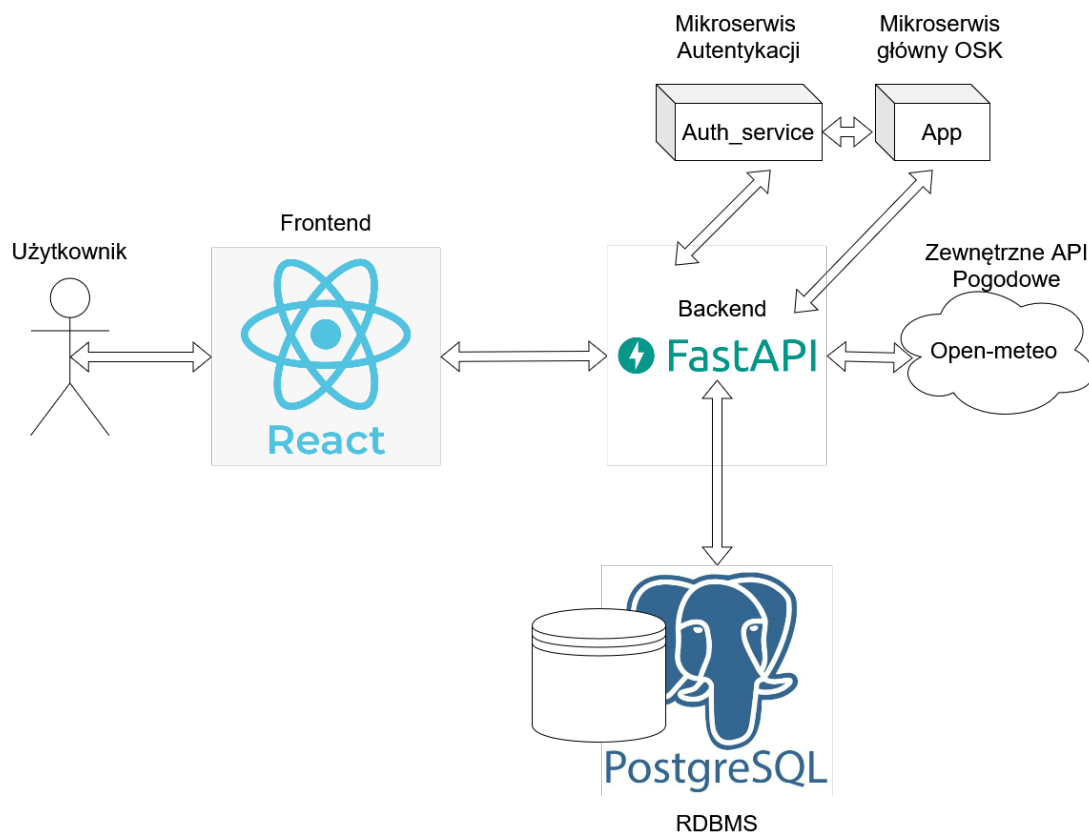
Niniejszy rozdział przedstawia szczegóły dotyczące założeń projektowych aplikacji – architektury, przypadków użycia, modeli danych i potencjalnych zagrożeń.

2.1. Architektura systemu

Rysunek 2.1 przedstawia schemat architektury rozwiązania z uwzględnieniem podziału na komponenty oraz wykorzystane technologie. System został zaprojektowany w oparciu o architekturę usługową, co zapewnia skalowalność, przejrzystość kodu oraz separację odpowiedzialności. Przepływ danych w aplikacji realizowany jest poprzez następujące warstwy:

- **Warstwa prezentacji:** Zaimplementowana przy użyciu biblioteki **React**. Odpowiada za interakcję z użytkownikiem oraz renderowanie interfejsu graficznego. Warstwa jest oddzielona od logiki biznesowej i komunikuje się z serwerem za pomocą żądań HTTP.
- **Serwer aplikacji:** Oparty na frameworku **FastAPI** (Python). Pełni rolę punktu komunikacyjnego. Jego zadaniem jest odbieranie zapytań z frontendu, walidacja danych wejściowych, komunikacja z bazą danych oraz zarządzanie ruchem do pozostałych usług.
- **Mikroserwisy wewnętrzne:** System wydziela funkcjonalności do niezależnych serwisów. Głównym z nich jest serwis aplikacyjny odpowiadający za całą logikę biznesową aplikacji. Drugim z elementów jest **aikroserwis autoryzacji (auth_service)**, który zarządza procesem logowania, wydawaniem tokenów sesyjnych oraz kontrolą dostępu na podstawie ról.

- **System zarządzania bazą danych RDBMS:** Jako system bazodanowy wykorzystano **PostgreSQL**. Przechowuje on ustrukturyzowane informacje. Dostęp do bazy jest ściśle izolowany - komunikuje się z backendem, co chroni dane przed bezpośrednim atakiem z zewnątrz.
- **Integracja z Zewnętrznym API (Open-meteo):** Aplikacja pobiera dodatkowe dane (informacje pogodowe, istotne w kontekście planowania jazdy) z zewnętrznej usługi chmurowej. Ze względów bezpieczeństwa, zapytania do API są realizowane z poziomu backendu.

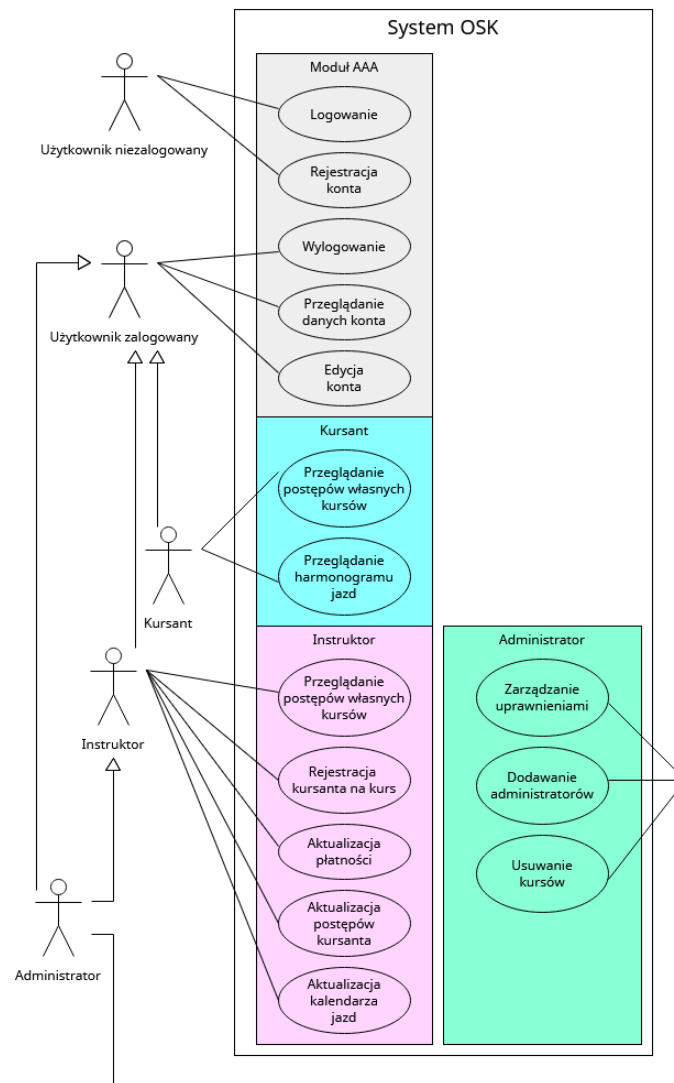


Rys. 2.1. Schemat architektury systemu

2.2. Kontrola dostępu (RBAC) - Przypadki użycia

W systemie zaimplementowano kontrolę dostępu opartą na rolach (ang. Role-Based Access Control). Zdefiniowano cztery główne role: użytkownik niezalogowany, kursant, instruktor oraz administrator. Rysunek 2.2 przedstawia diagram przypadków użycia, który obrazuje podział uprawnień oraz powiązania między poszczególnymi aktorami a funkcjonalnościami systemu. Szczegółowy podział odpowiedzialności w ramach zaimplementowanego modelu RBAC prezentuje się następująco:

- **Użytkownik niezalogowany:** Posiada dostęp wyłącznie do operacji wstępnych: rejestracji nowego konta oraz logowania.
- **Kursant:** Ma możliwość podglądu własnego harmonogramu zaplanowanych lekcji oraz bieżących postępów w kursie.
- **Instruktor:** Posiada uprawnienia operacyjne w ramach przypisanych mu kursantów. Odpowiada za zarządzanie kalendarzem, uzupełnianie liczby godzin wyjeżdżonych przez kursantów oraz modyfikację swoich specjalizacji.
- **Administrator:** Dziedziczy wszystkie możliwości instruktora, rozszerzając je o uprawnienia globalne oraz destrukcyjne. Do jego zadań należy zarządzanie uprawnieniami (tworzenie nowych kont administracyjnych, modyfikacja ról) oraz możliwość usuwania kursów w przypadku błędów.

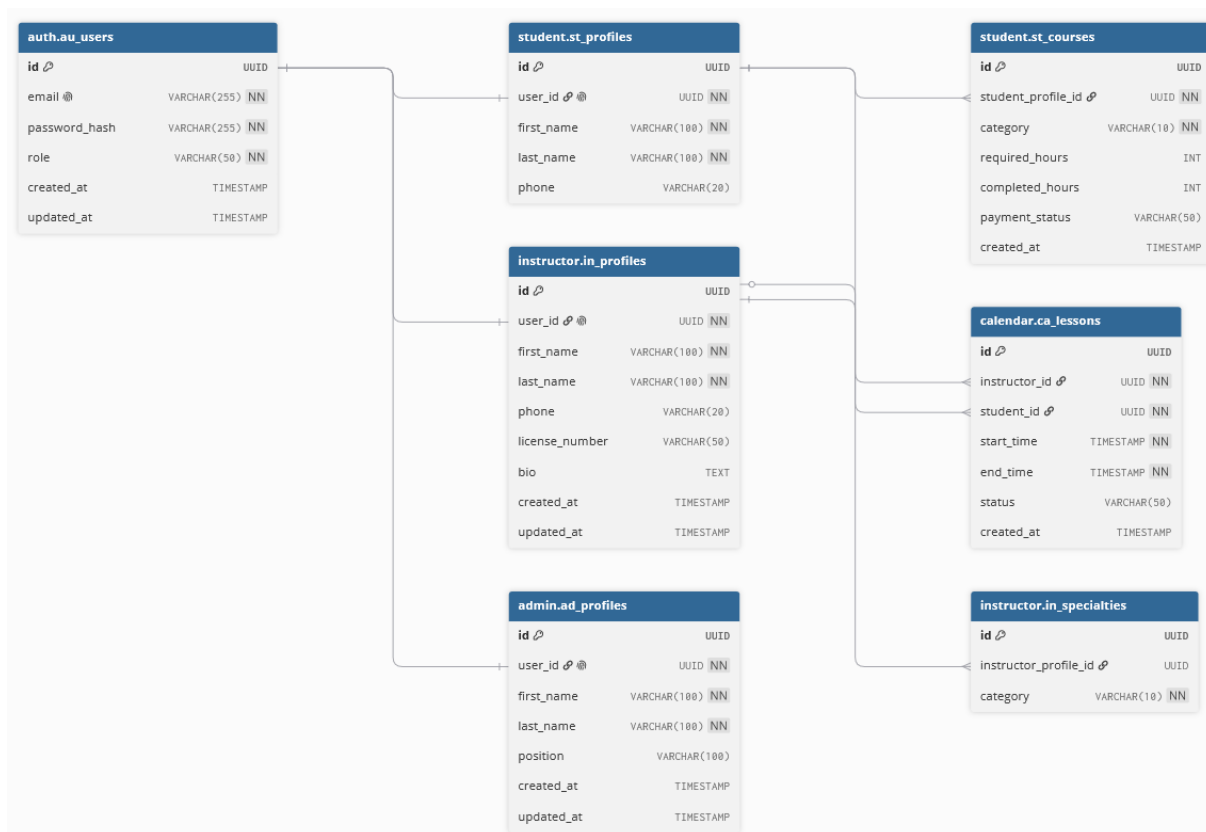


Rys. 2.2. Diagram przypadków użycia z uwzględnieniem ról

2.3. Model danych

Do przechowywania danych wykorzystano relacyjną bazę danych składającą się z tabel podzielonych na pięć modułów: autentykacji, kursanta, instruktora, kalendarza oraz administratora. Na rysunku 2.3 przedstawiono diagram związków encji bazy danych. Poniżej przedstawiono charakterystykę poszczególnych tabel oraz zakres przechowywanych w nich danych:

- **auth.au_users:** Główna tabela odpowiedzialna za autentykację i kontrolę dostępu (RBAC). Przechowuje unikalne adresy e-mail, skróty haseł oraz przypisane role użytkowników (KURSANT, INSTRUKTOR, ADMIN).
- **student.st_profiles:** Zawiera podstawowe dane osobowe kursantów, takie jak imię, nazwisko i numer telefonu. Jest powiązana relacją jeden do jednego z kontem użytkownika w module autentykacji.
- **student.st_courses:** Odpowiada za śledzenie postępów szkoleniowych. Przechowuje informacje o wybranej kategorii prawa jazdy (AM, B, C), liczbie wymaganych i wyjeżdżonych godzin oraz statusie płatności za kurs.
- **instructor.in_profiles:** Przechowuje szczegółowe dane profilowe instruktorów, w tym imię, nazwisko, telefon, numer posiadanej licencji oraz opcjonalny biogram.
- **instructor.in_specialties:** Tabela pomocnicza określająca specjalizacje instruktorów, czyli kategorie prawa jazdy, w ramach których dany instruktor ma uprawnienia do prowadzenia szkoleń.
- **calendar.ca_lessons:** Odzwierciedla harmonogram zaplanowanych jazd. Łączy profile instruktorów i kursantów, przechowując informacje o czasie rozpoczęcia i zakończenia lekcji oraz jej bieżącym statusie (np. SCHEDULED).
- **admin.ad_profiles:** Zawiera profile osób zarządzających systemem, przechowując ich imię, nazwisko oraz zajmowane w strukturze organizacji stanowisko.



Rys. 2.3. Diagram związków encji (ERD)

2.4. Potencjalne zagrożenia

Na etapie projektowania zidentyfikowano następujące potencjalne zagrożenia dla bezpieczeństwa aplikacji, opierając się na aktualnej klasyfikacji OWASP Top 10 [1]:

- **Naruszenie kontroli dostępu (Broken Access Control – OWASP A01:2025):**
Z uwagi na złożoną strukturę ról (Kursant, Instruktor, Administrator), zagrożeniem jest pozioma i pionowa eskalacja uprawnień. Brak rygorystycznej weryfikacji roli na każdym punkcie końcowym backendu mógłby pozwolić np. kursantowi na samodzielną modyfikację wyjeżdżonych godzin lub instruktorowi na nieuprawnione usunięcie kursu, co jest wyłącznym prawem administratora.
- **Błędy uwierzytelniania (Authentication Failures – OWASP A07:2025):**
Zagrożenie może pojawić się w przypadku braku mechanizmów obronnych w protokole uwierzytelniania. W przypadku braku zabezpieczenia ograniczania liczby żądań na punktach wejścia do aplikacji, system będzie podatny na ataki automatyczne (np. *brute-force* lub *credential stuffing*). Ponadto, zagrożenie stanowią potencjalne błędy w cyklu życia sesji użytkownika. Brak bezpiecznego unieważnia-

nia sesji po stronie serwera oraz brak rozróżnienia między krótkotrwałym tokenem dostępowym a długotrwałym tokenem odświeżającym, co w fazie operacyjnej drastycznie zwiększa ryzyko związane z przejęciem aktywnego tokenu.

- **Wstrzykiwanie danych (Injection – OWASP A05:2021) oraz XSS:** Zagrożenie może pojawić się w sytuacji, gdy architektura systemu dopuszczałaby przekazywanie niefiltrowanych danych pochodzących od użytkownika bezpośrednio do interpreterów poleceń lub silników bazodanowych. Brak walidacji i parametryzacji zapytań na poziomie komunikacji z bazą danych mogłoby stworzyć ryzyko wykonania złośliwego kodu (np. *SQL Injection*). Brak neutralizacji kodu wykonywalnego w warstwie prezentacji może skutkować wstrzyknięciem złośliwych skryptów wykonywanych po stronie klienta (ataki typu *Cross-Site Scripting – XSS*).

3. Software supply chain failures

3.1. Zestawienie wykorzystanych technologii

W tabeli 3.1 przedstawiono wykaz narzędzi i frameworków użytych w projekcie wraz z ich wersjami.

3.2. Analiza historycznych podatności

Mając na uwadze bezpieczeństwo (Security) oraz niezawodność (Safety), przeanalizowano luki występujące w poprzednich wersjach użytych narzędzi, aby udowodnić odporność obecnego stosu technologicznego:

- 1) **Podatność 1** (React w wersjach 19.0.0, 19.1.0, 19.1.1, and 19.2.0): Identyfikator CVE-2025-55182 (tzw. React2Shell, oznaczona jako krytyczna, CVSS 10.0). Luka polegała na niebezpiecznej deserializacji danych w architekturze React Server Components (RSC), co pozwalało atakującemu na zdalne wykonanie dowolnego kodu (RCE) na serwerze za pomocą jednego spreparowanego żądania HTTP. Wybrana wersja (**19.0.1**) łąta ten problem, wprowadzając bezpieczniejsze parsowanie i zamykając tę drogę ataku [2].
- 2) **Podatność 2** (FastAPI w wersjach poniżej 0.65.2): Identyfikator CVE-2021-32677. Podatność związana z wadliwym mechanizmem uwierzytelniania. W starszych wersjach frameworka endpointy opierające autoryzację na ciasteczkach, które jednocześnie przyjmowały ładunek payload w formacie JSON, mogły zostać oszukane przez atakującego, pozwalając na całkowite ominięcie autoryzacji. Wersja 0.136.1 zawiera w pełni naprawioną logikę weryfikacji [3].
- 3) **Podatność 3** (PostgreSQL w wersjach poniżej 18.4, 17.10, 16.14, **15.18 (wybrana w projekcie)**, 14.23): Identyfikator CVE-2026-6472. Poważny błąd walidacji uprawnień (CVSS 8.8) związany z poleceniem CREATE TYPE. Brak odpowiedniej autoryzacji pozwalał twórcy obiektu na przejęcie innych zapytań wykorzystujących zmienną `search_path`, co w efekcie umożliwiało wykonanie złośliwego kodu SQL (Arbitrary SQL Execution). Wdrażana w tym projekcie zaktualizowana wersja 15.18 eliminuje ten wektor ataku poprzez rygorystyczne sprawdzanie uprawnień (CREATE privilege) [4].

Narzędzie / Framework	Wersja	Krótki opis zastosowania
Node.js	24.x	Środowisko uruchomieniowe niezbędne do działania narzędzi frontendowych oraz procesu budowania aplikacji.
TypeScript	5.8.2	Język dodający statyczne typowanie do JavaScriptu.
React	19.0.1	Główna biblioteka wykorzystana do budowy interfejsu użytkownika - Frontend.
react-router-dom	7.15.0	Biblioteka zarządzająca routingiem po stronie przeglądarki w architekturze Single Page Application.
Vite	6.2.3	Narzędzie używane do budowania oraz serwowania warstwy frontendowej.
Tailwind CSS	4.1.14	Framework CSS użyty do zarządzania warstwą wizualną aplikacji.
Axios	1.17.0	Klient HTTP wykorzystywany we frontendzie do asynchronicznej i bezpiecznej komunikacji z backendem.
Vitest	4.1.8	Framework do testów automatycznych na frontendzie.
FastAPI	0.136.1	Framework Pythona do budowy logiki biznesowej i wystawiania punktów końcowych REST API.
Uvicorn	0.28.0	Serwer ASGI używany do uruchamiania aplikacji FastAPI, zapewniający wydajność obsługi żądań sieciowych.
Pydantic	2.13.4	Biblioteka do walidacji i parsowania danych wejściowych w backendzie.
PostgreSQL	15.18	Relacyjna baza danych przechowująca ustrukturyzowane informacje.
SQLAlchemy	2.0.49	Biblioteka ORM zapewniająca wyższy poziom abstrakcji oraz automatyczną ochronę przed atakami typu SQL Injection.
psycopg	3.1.18	Adapter bazy danych PostgreSQL dla języka Python, realizujący fizyczne połączenie z bazą.
passlib / python-jose	1.7.4 / 3.3.0	Narzędzia kryptograficzne służące do bezpiecznego hashowania haseł oraz zarządzania tokenami JWT.
python-multipart	0.0.9	Pakiet do obsługi danych z formularzy, potrzebny przy standardowym logowaniu OAuth2 w systemie.
pytest	8.0.0	Framework do wykonywania testów niezawodności i stabilności systemu na backendzie.
httpx	0.28.1	Asynchroniczny klient HTTP wykorzystywany na backendzie do komunikacji z wewnętrznymi i zewnętrznymi usługami.

Tabela 3.1. Zestawienie wykorzystanych technologii i bibliotek

4. Implementacja aplikacji

W niniejszym rozdziale omówiono wybrane elementy zaimplementowanego systemu pod kątem bezpieczeństwa oraz niezawodności. Zidentyfikowano wdrożone mechanizmy ochrony, a także wskazano braki oraz przypadki niebezpiecznych implementacji lub nieodpowiedniej obsługi błędów, które wymagają poprawy w przyszłości.

4.1. Autentykacja, zarządzanie hasłami i RBAC

W systemie zaimplementowano mechanizm logowania i rejestracji oparty na tokenach JWT. W celu ochrony hasel użytkowników przed wyciekiem zastosowano algorytm haszujący Argon2. Dodatkowo system wymusza kontrolę dostępu opartą na rolach (RBAC).

Listing 4.1. Fragment weryfikacji logowania w `auth_service/router.py`

```
1 if not user or not utils.verify_password(user_credentials.password, user.password_hash
2 ):
3     raise HTTPException(
4         status_code=status.HTTP_401_UNAUTHORIZED,
5         detail="Incorrect credentials."
```

Aspekty bezpieczeństwa i niezawodności:

- **Zrealizowano (Security):** Użycie ujednoliconego komunikatu o błędzie (*Incorrect credentials*) bez względu na to, czy nie istnieje użytkownik, czy podano złe hasło, zapobiega atakom typu *User Enumeration* (odgadywanie istniejących adresów email). Ponadto tokeny JWT mają ustawiony krótki czas wygasania (30 minut).
- **Braki i ulepszenia:** W systemie brakuje zabezpieczenia przed atakami typu *brute-force* (brak mechanizmu *Rate Limiting*). Nie ma również obsługi unieważniania tokenów przed czasem (np. czarnej listy tokenów przy wylogowaniu).

4.2. Autoryzacja i ochrona zasobów

Zaimplementowano weryfikację uprawnień na poziomie dostępu do konkretnych zasobów (*Horizontal Authorization*). Listing 4.2 prezentuje przykładowe ograniczenie dostępu do lekcji w module kalendarza – system upewnia się, że instruktor może odczytywać wyłącznie własne zaplanowane jazdy.

Listing 4.2. Weryfikacja uprawnień instruktora w żądaniu HTTP

```
1 if user_role == "instructor":
```

```

2 caller_profile = db.query(models.InProfiles).filter(models.InProfiles.user_id ==
user_id).first()
3 if not caller_profile or caller_profile.id != instructor_profile_id:
4     raise HTTPException(
5         status_code=status.HTTP_403_FORBIDDEN,
6         detail="Access restricted to owner, instructor or platform administrator."
7     )

```

Aspekty bezpieczeństwa i niezawodności:

- **Zrealizowano (Security):** Skutecznie zabezpieczono system przed krytycznymi atakami typu IDOR (*Insecure Direct Object Reference*). Atakujący, będący zalogowanym instruktorem, nie jest w stanie odczytać kalendarza innego instruktora przez zmianę parametru `instructor_profile_id` w żądaniu. System poprawnie rzuca wyjątek z kodem 403 Forbidden.
- **Braki i ulepszenia:** Logika autoryzacyjna zawiera tzw. *hardcoding* ról (sztywne porównanie `user_role == "instructor"`). Wraz ze wzrostem złożoności systemu i dodawaniem nowych ról, zarządzanie takimi instrukcjami warunkowymi będzie podatne na błędy. Można to wydzielić do reużywalnych, zewnętrznych zależności polityk dostępu.

4.3. Komunikacja między mikroservisami i weryfikacja tokenów

Architektura aplikacji wymaga komunikacji między głównym backendem a mikroservisem autoryzacji. Pobieranie danych o zalogowanym użytkowniku oparto o standard OAuth2PasswordBearer oraz klienta `httpx`. Przykład komunikacji przedstawiono w listingu 4.3.

Listing 4.3. Asynchroniczna komunikacja i obsługa błędów mikroservis Auth

```

1 async def get_user_from_auth(token: str):
2     async with httpx.AsyncClient() as client:
3         try:
4             response = await client.get(
5                 "http://localhost:8001/auth/verify",
6                 headers={"Authorization": f"Bearer {token}"})
7         except:
8             if response.status_code == 200:
9                 return response.json()
10            return None
11     except httpx.RequestError:
12         raise HTTPException(
13             status_code=status.HTTP_503_SERVICE_UNAVAILABLE,
14             detail="Authentication service is currently unavailable."
15         )

```

Aspekty bezpieczeństwa i niezawodności:

- **Zrealizowano (Security):** System bezpiecznie hermetyzuje token sesyjny, wstrzykując go do nagłówka żądania (`Authorization: Bearer`). W przypadku braku autoryzacji aplikacja rzuca wyjątek z kodem 401 Unauthorized.
- **Zrealizowano (Safety):** Pomyślano o awarii infrastruktury. Jeśli mikroserwis autentykacji ulegnie awarii i przestanie nasłuchiwać, główny backend nie zamyka się z krytycznym błędem 500, lecz przechwytuje `httpx.RequestError` i informuje o niedostępności usługi (503 Service Unavailable).
- **Braki i ulepszenia (Safety):** Wywołanie sieciowe nie posiada zdefiniowanego limitu czasu oczekiwania (`timeout`). Jeśli usługa Auth będzie utrzymywać otwarte połączenia ale nie zwracać odpowiedzi, wszystkie zapytania do głównego backendu zaczną blokować wątki. Może to doprowadzić do zamrożenia głównej aplikacji. Zabezpieczyć można to, dodając parametr `timeout=5.0` do klienta. Dodatkowo adres URL (`localhost:8001`) nie powinien znajdować się bezpośrednio w kodzie, lecz być pobierany ze zmiennej środowiskowej.

4.4. Integracja z zewnętrznym API pogodowym

Aplikacja pobiera dane pogodowe z zewnętrznego serwisu Open-Meteo. Zaprojektowano podstawową obsługę błędów sieciowych za pomocą bloku `try-except`. Listing 4.4 przedstawia obsługę połączenia z zewnętrznym API.

Listing 4.4. Obsługa połączenia z zewnętrznym API w `weather_service.py`

```

1 try:
2     async with httpx.AsyncClient() as client:
3         response = await client.get(OPEN_METEO_URL, params=params)
4         response.raise_for_status()
5         # (...) parsowanie danych
6 except httpx.HTTPError:
7     return None

```

Aspekty bezpieczeństwa i niezawodności:

- **Zrealizowano (Safety):** W przypadku awarii zewnętrznego API lub błędu HTTP (np. 500, 404), aplikacja łapie wyjątek `httpx.HTTPError` i zwraca wartość `None`.
- **Braki i ulepszenia:** Zaimplementowana obsługa wyjątków może być niewystarczająca. W kliencie HTTP **nie zdefiniowano limitu czasu (`timeout`)**. Jeśli zewnętrzne API przestanie odpowiadać i połączenie będzie „wisieć”, serwer może wyczerpać dostępne wątki i sam przestanie odpowiadać użytkownikom. Należy wprowadzić `timeout` oraz mechanizm *Circuit Breaker*.

4.5. Warstwa bazy danych (Security & Safety)

Aplikacja łączy się z systemem PostgreSQL z wykorzystaniem narzędzia ORM (SQLAlchemy).

Listing 4.5. Inicjalizacja i zamykanie sesji w `database.py`

```
1 SQLALCHEMY_DATABASE_URL = "postgresql+psycopg://root:rootpassword@localhost:5432/  
  osk_default"  
2 # (...)  
3 def get_db():  
4     db = SessionLocal()  
5     try:  
6         yield db  
7     finally:  
8         db.close()
```

Aspekty bezpieczeństwa i niezawodności:

- **Zrealizowano (Security & Safety):** Użycie ORM uodparnia system na najpopularniejsze ataki *SQL Injection*. Blok `finally: db.close()` gwarantuje, że połączenia z bazą są zwalniane do puli nawet w przypadku wystąpienia błędu w aplikacji (zapobiega wyciekom pamięci/połączeń).
- **Braki i ulepszenia (Security):** Poważną luką bezpieczeństwa jest trzymanie danych logowania (`root:rootpassword`) bezpośrednio w kodzie źródłowym (tzw. *hardcoding*). Parametry te powinny być wstrzykiwane bezpiecznie przez zmienne środowiskowe (`.env`).
- **Braki i ulepszenia (Safety):** system nie jest przygotowany na awarię głównej bazy danych. Brak nawiązania połączenia spowoduje natychmiastowe wyrzucenie błędu krytycznego (np. *OperationalError*) bezpośrednio do użytkownika. Brakuje mechanizmu ponawiania prób połączenia (ang. *retry*) oraz odpowiedniego opakowania błędów bazy w wyjątki HTTP (np. 503 Service Unavailable).

4.6. Implementacja testów automatycznych

W celu weryfikacji mechanizmów aplikacji, system został pokryty zestawem zautomatyzowanych testów przygotowanych przy użyciu frameworku `pytest` oraz asynchronicznego klienta `httpx`. Scenariusze testowe podzielono na dwa obszary: testy izolowane mikroserwisu autoryzacji oraz testy integracyjne całego systemu.

4.6.1. Izolacja środowiska testowego

Aby zapobiec nadpisywaniu lub modyfikacji realnych danych podczas wykonywania testów, zaimplementowano mechanizm izolacji bazy danych. Tworzone są zagnieżdżone transakcje, które po zakończeniu testu są wycofywane. Gwarantuje to, że każdy test uruchamia się na czystym stanie i nie wpływa na inne scenariusze.

4.6.2. Testowanie mikroserwisu autentykacji

W ramach osobnego zbioru testów (`test_auth.py`) sprawdzana jest logika uwierzytelniania. Zweryfikowano w nich następujące przypadki:

- **Poprawność rejestracji i logowania:** Sprawdzenie, czy nowo zarejestrowany kursant może się poprawnie zalogować i czy otrzymuje ważny token JWT.
- **Ochrona ról (RBAC):** Próba zalogowania się z użyciem poprawnego hasła, ale zadeklarowaniem błędnej roli (np. kursant próbujący zalogować się jako administrator). Test weryfikuje, czy system odrzuci takie żądanie z kodem 401 Unauthorized.
- **Zabezpieczenie kont uprzywilejowanych:** Weryfikacja, czy utworzenie nowego profilu o randze `admin` jest niemożliwe z poziomu niezalogowanego użytkownika, a udaje się jedynie wtedy, gdy żądanie posiada w nagłówku token obecnego administratora.

4.6.3. Testy integracyjne przepływu pracy

W pliku `test_core_integration.py` zaimplementowano testy sprawdzające poprawność komunikacji pomiędzy dwoma mikroserwisami (Głównym API oraz serwisem autoryzacji). Zrealizowane scenariusze weryfikują:

- Pełny przepływ (ang. *workflow*): od wstępnej rejestracji, poprzez logowanie w serwisie Auth i pobranie tokenu, po uzupełnienie profilu instruktora w głównym serwisie.
- Poprawność tworzenia powiązań w bazie danych, np. weryfikacja czy system pozwala instruktorowi zaplanować lekcję dla konkretnego kursanta na określoną godzinę i czy zwraca status 200 OK.
- **Ochronę przed eskalacją uprawnień:** Sprawdzenie podatności na modyfikację danych w innych modułach.

W listingu 4.6 przedstawiono fragment testu integracyjnego, który sprawdza skuteczność zabezpieczeń autoryzacyjnych. Test upewnia się, że użytkownik posiadający je-

dynie token kursanta nie ma prawa dostępu do endpointu umożliwiającego edycję profilu instruktora, uodparniając system na ataki manipulacji rolami.

Listing 4.6. Test zabezpieczeń przed eskalacją uprawnień na poziomie integracji

```
1 def test_10_role_mismatch_protection():
2     # Uzycie tokenu kursanta
3     headers = {"Authorization": f"Bearer {state['student_token']}"}
4     profile_payload = {
5         "first_name": "Malicious",
6         "last_name": "Actor",
7         "phone": "+48000000000",
8         "license_number": "BAD-PROMPT",
9         "bio": "Trying to bypass role restrictions."
10    }
11    # Proba wyslania zadan do modulu instruktora
12    response = httpx.post(f"{CORE_URL}/instructor/register_full", json=profile_payload
13    , headers=headers)
14
15    # System powinien odrzucic to zadanie ze wzgledu na brak uprawnień
16    assert response.status_code == 403
```


5. Wdrożenie projektu

W tym rozdziale opisano proces wdrożenia aplikacji. Do poprawnego uruchomienia całego środowiska deweloperskiego wymagane jest posiadanie zainstalowanych narzędzi: Docker, Python w wersji 3.12 oraz Node.js z menedżerem pakietów npm.

5.1. Środowisko bazodanowe (PostgreSQL i pgAdmin)

W celu zapewnienia spójnego środowiska bazy danych dla wszystkich programistów, wykorzystano konteneryzację za pomocą narzędzia Docker. Plik `docker-compose.yml` zawiera konfigurację kontenerów oraz uruchamia instancję bazy PostgreSQL oraz interfejs do jej zarządzania – pgAdmin.

Aby uruchomić kontenery w tle, należy w głównym katalogu projektu wywołać polecenie:

```
1 docker-compose up --build -d
```

Podczas pierwszego uruchomienia bazy, skrypt `init.sql` zamontowany w kontenerze zostanie wykonany automatycznie, tworząc wymagane schematy (m.in. `auth`, `student`, `instructor`, `calendar`) oraz tabele.

Po uruchomieniu środowiska, interfejs pgAdmin dostępny jest w przeglądarce pod adresem `http://localhost:5050` (logowanie: `admin@osk.pl` / `admin`). Ponadto przygotowano skrypt `data.sql`, który pozwala na zasilenie bazy testowymi danymi. Można go zaimportować i wykonać bezpośrednio z poziomu interfejsu pgAdmin.

5.2. Serwer backend (FastAPI)

Część serwerowa przedstawionej aplikacji została napisana w języku Python przy użyciu frameworku FastAPI. W celu jej uruchomienia zaleca się utworzenie wirtualnego środowiska przed zainstalowaniem wszelkich zależności. W celu ułatwienia procesu konfiguracji i uruchomienia aplikacji, utworzono skrypty `run.ps1` (dla systemów Microsoft Windows) oraz `run.sh` (dla sytemów Linux). Backend aplikacji został podzielony na dwa osobne serwisy. Aby je uruchomić należy przejść do katalogu projektu, uruchomić terminal i wpisać odpowiednią komendę wywołującą skrypt uruchamiający. W listingu 5.1 przedstawiono uruchomienie głównego serwisu, a w listingu 5.2 – uruchomienie mikroservisu autoryzacji.

Listing 5.1. Inicjalizacja i uruchomienie serwera backendowego – serwis główny

```
1 ## W głównym katalogu projektu
```

```
2 # System Windows
3 cd backend/app
4 ./run.ps1
5
6 # System Linux
7 cd backend/app
8 ./run.sh
```

Listing 5.2. Inicjalizacja i uruchomienie serwera backendowego – mikroserwis autoryzacji

```
1 ## W głównym katalogu projektu
2 # System Windows
3 cd backend/auth_service
4 ./run.ps1
5
6 # System Linux
7 cd backend/auth_service
8 ./run.sh
```

5.3. Aplikacja Frontend (React)

Warstwa prezentacji bazuje na bibliotece React. Aby przygotować aplikację kliencką do działania, należy przejść do katalogu projektu i za pomocą terminala zainstalować niezbędne zależności przy użyciu menedżera pakietów Node (listing 5.3.).

Listing 5.3. Instalacja zależności i uruchomienie aplikacji React

```
1 # Przejście do katalogu frontend
2 cd frontend
3
4 # Pobranie i instalacja paczek
5 npm install
6
7 # Uruchomienie serwera frontendu
8 npm run dev
```

Po poprawnym uruchomieniu skryptu, aplikacja połączy się z lokalnym środowiskiem Core API i będzie dostępna z poziomu przeglądarki internetowej pod adresem wskazanym w konsoli (<http://localhost:3000>).

5.4. Testy aplikacji (pytest)

Część testowa przedstawionej aplikacji została przygotowana w języku Python przy użyciu frameworku `pytest`. W celu weryfikacji poprawności działania kodu zaleca się uruchamianie skryptów testowych wewnątrz wcześniej stworzonego wirtualnego środowiska, po uprzednim zainstalowaniu pakietów testowych. W celu automatyzacji oraz ułatwienia procesu uruchamiania testów jednostkowych i integracyjnych dla obu mikroserwisów, utworzono zbiorcze skrypty `tests.ps1` (dla systemów Microsoft Windows) oraz `tests.sh` (dla systemów Linux). Skrypty te automatycznie wyszukują i wykonują testy przygotowane

zarówno dla głównego serwisu aplikacji, jak i mikroservisu autoryzacji. Aby przeprowadzić procedurę testową, należy przejść do katalogu backendu, uruchomić terminal i wpisać odpowiednią komendę. W listingu 5.4 przedstawiono sposób wywołania automatycznych testów.

Listing 5.4. Uruchomienie automatycznych testów jednostkowych i integracyjnych

```
1 ## W głównym katalogu projektu
2 # System Windows
3 cd backend
4 ./tests.ps1
5
6 # System Linux
7 cd backend
8 ./tests.sh
```

6. Podsumowanie i wnioski końcowe

Projekt pozwolił na praktyczne zrozumienie wyzwań związanych z bezpieczeństwem aplikacji webowych. W ramach projektu przeanalizowano zagrożenia związane z założeniami systemu oraz zastosowanymi narzędziami, bibliotekami i frameworkami. Uzasadniono wykorzystanie konkretnych wersji technologii, biorąc pod uwagę przeszłe podatności. Przedstawiono implementację aplikacji z uwzględnieniem ważnych mechanizmów wpływających na bezpieczeństwo i niezawodność systemu. Zidentyfikowano obszary wymagające dalszych prac w celu poprawienia bezpieczeństwa i niezawodności systemu. Wdrożone zautomatyzowane środowisko testowe pozwoliło na sprawdzenie szczelności i ochrony systemu przed atakami lub próbami eskalacji uprawnień. Mimo wdrożenia wielu zabezpieczeń, zidentyfikowano również obszary wymagające dalszych prac. Dodatkowo zaprezentowano scenariusz wdrożenia aplikacji, wzorując się na sprawdzonym mechanizmie wykorzystywanym w trakcie jej budowy. Utworzenie dedykowanych skryptów automatyzujących stanowi ułatwienie dla procesu ciągłej integracji i wdrażania (CI/CD). Ostatecznie, zastosowanie podziału na niezależne mikroservisy oraz ścisła kontrola dostępu oparta na rolach (RBAC) wyraźnie zwiększyły elastyczność, skalowalność i ogólny poziom bezpieczeństwa całego systemu.

Literatura

- [1] OWASP Top 10 2025. <https://owasp.org/Top10/2025/>. Dostęp 09.06.2026.
- [2] Podatność frameworku React CVE-2025-55182 <https://nvd.nist.gov/vuln/detail/CVE-2025-55182>. Dostęp 09.06.2026.
- [3] Podatność frameworku FastAPI CVE-2021-32677 <https://nvd.nist.gov/vuln/detail/CVE-2021-32677>. Dostęp 09.06.2026.
- [4] Podatność bazy danych PostgreSQL CVE-2026-6472 <https://nvd.nist.gov/vuln/detail/CVE-2026-6472>. Dostęp 09.06.2026.